

CSE401: Numerical Analysis, Simulation and Modeling

Random Number Generation and Monte Carlo Simulation

Nafis Tahmid

Department of CSE, BUET

Why Do We Need Randomness?

A Very Serious Question

Why would a **computer scientist**
want to use **random numbers**?

A Very Serious Question

Because the **real world**
refuses to be deterministic.

The Real World Is Messy

Consider these questions:

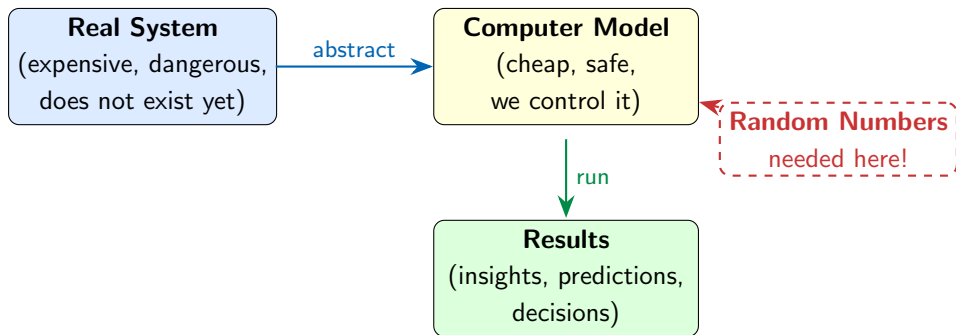
- How long will a customer wait in line at the bank?
- Will this bridge survive an earthquake?
- How many servers does Amazon need on Black Friday?
- Will this new drug actually work?

The Problem

We cannot solve these with a neat formula. We cannot always build the real thing and test it.

Solution: We **simulate** — we build a computer model and let it run.

What Is Simulation?



The model needs **randomness** to mimic the unpredictability of the real world.

Where Random Numbers Show Up

- **Customer arrivals:** When does the next customer walk in? (Random!)
- **Service times:** How long does each haircut take? (Random!)
- **Component failures:** When does a part break? (Random!)
- **Weather, traffic, stock prices:** (All random!)

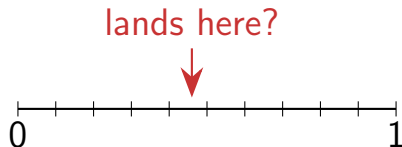
Key Idea

To simulate any system with uncertainty, we first need a reliable source of **random numbers** between 0 and 1. Everything else is built on top of that.

Properties of Random Numbers

What IS a Random Number?

Imagine you have a magic spinner...



It points to some value between 0 and 1.

Any value is equally likely.

What IS a Random Number?

That is a **random number**!

A value R drawn from the interval $[0, 1]$
where **every point is equally likely**.

The Two Sacred Properties

A sequence of random numbers $R_1, R_2, R_3, \dots$ must have:

Property 1: Uniformity

Each R_i is equally likely to fall anywhere in $[0, 1]$.

No part of $[0, 1]$ is “preferred.”

The number 0.73 is just as likely as 0.12.

Property 2: Independence

Knowing R_1 tells you **nothing** about R_2 .

If $R_1 = 0.9$, that does not make R_2 more likely to be high or low.

Each number minds its own business.

Both properties must hold **simultaneously**.

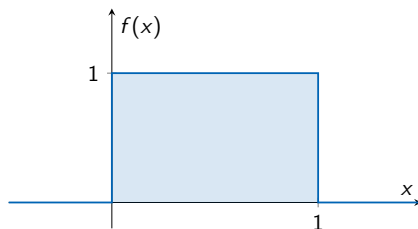
Uniformity without independence (or vice versa) is not enough.

The Uniform Distribution $U(0, 1)$

Mathematically, each R_i follows a **continuous uniform distribution** on $[0, 1]$.

Its probability density function (pdf) is:

$$f(x) = \begin{cases} 1, & 0 \leq x \leq 1 \\ 0, & \text{otherwise} \end{cases}$$



It is flat. Boring. **Perfectly fair.** That is exactly what we want.

What Is the Average Random Number?

Let us compute $E(R)$, the **expected value** (average) of a random number.

$$\begin{aligned} E(R) &= \int_0^1 x \cdot f(x) \, dx \\ &= \int_0^1 x \cdot 1 \, dx \quad (\text{since } f(x) = 1 \text{ on } [0, 1]) \\ &= \left[\frac{x^2}{2} \right]_0^1 \\ &= \frac{1}{2} - 0 = \boxed{\frac{1}{2}} \end{aligned}$$

Not shocking. The average of numbers between 0 and 1 is $\frac{1}{2}$.

(Intuition still works!)

How “Spread Out” Are They?

The **variance** measures how spread out the values are.

$$\text{Var}(R) = E(R^2) - [E(R)]^2$$

First, compute $E(R^2)$:

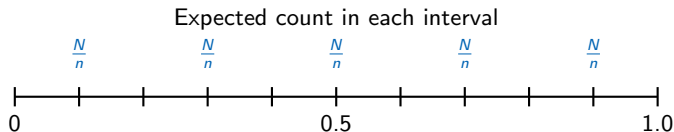
$$E(R^2) = \int_0^1 x^2 \cdot 1 \, dx = \left[\frac{x^3}{3} \right]_0^1 = \frac{1}{3}$$

Now plug in:

$$\begin{aligned} \text{Var}(R) &= \frac{1}{3} - \left(\frac{1}{2} \right)^2 = \frac{1}{3} - \frac{1}{4} \\ &= \frac{4 - 3}{12} = \boxed{\frac{1}{12}} \approx 0.0833 \end{aligned}$$

Consequences of Uniformity and Independence

If we generate N random numbers and divide $[0, 1]$ into n equal subintervals:



1. **Uniformity** means the expected number in each subinterval is N/n .
2. **Independence** means falling in one interval has no effect on where the next number lands.

Generating Pseudo-Random Numbers

A Little Secret

Computers are **deterministic** machines.

They do exactly what you tell them.
Every. Single. Time.

A Little Secret

So how can a deterministic machine
produce **random** numbers?

Short answer: **It cannot.**

A Little Secret

That is why we call them
“pseudo-random” numbers

“Pseudo” = false, fake, imitation.

They *look* random. They *act* random.
But they are computed by a formula.

Why “Fake” Random Is Actually Fine

If the method is known, the sequence can be **repeated**.

Does that not mean it is not random?

Technically, yes. But **repeatability is actually useful**:

- **Debugging:** If your simulation crashes, you can reproduce the exact same run.
- **Comparison:** Test two system designs with the *exact same* random events.
- **Verification:** Someone else can replicate your results.

The Goal

Produce a sequence of numbers between 0 and 1 that **simulates** the ideal properties of uniformity and independence as closely as possible.

Things That Can Go Wrong

Pseudo-random number generators can have **defects**:

1. Numbers might not be **uniformly distributed**. (Some regions preferred)
2. Numbers might be **discrete** instead of continuous. (Gaps!)
3. The **mean** might be too high or too low. (Biased)
4. The **variance** might be wrong. (Too clustered or too spread)
5. There might be **dependence** between successive numbers. (Patterns!)

Good News

We can **test** for these problems. Generators that pass rigorous tests exist — no excuse for using bad ones.

What We Want in a Generator

A good pseudo-random number generator should be:

1. **Fast.** Simulations may need *millions* of random numbers.
2. **Portable.** Same results on different computers.
3. **Long period.** The sequence should not repeat too soon.
 - If you need 10,000 numbers, the period should be *much* longer.
 - A degenerate generator repeats the **same number** — unacceptable!
4. **Repeatable.** Given the same starting point, produce the same sequence.
5. **Statistically good.** Pass tests for uniformity and independence.

Inventing something that *looks* like it generates random numbers is easy.

Inventing something that *actually does it well* is incredibly difficult.

Linear Congruential Method (LCM)

One formula to rule them all:

$$X_{i+1} = (a \cdot X_i + c) \bmod m \quad i = 0, 1, 2, \dots$$

- X_0 = the **seed** (starting value — you choose this)
- a = the **multiplier** (a carefully chosen constant)
- c = the **increment** (another constant)
- m = the **modulus** (keeps numbers bounded)

Two Flavors

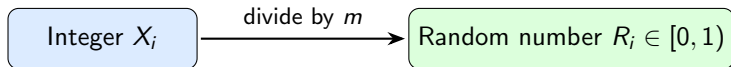
$c \neq 0$: **Mixed** congruential method. $c = 0$: **Multiplicative** congruential method.

From Integers to Random Numbers

The LCM generates **integers** $X_0, X_1, X_2, \dots$ in the range $\{0, 1, \dots, m-1\}$.

To get random numbers R_i between 0 and 1, we simply divide:

$$R_i = \frac{X_i}{m} \quad i = 1, 2, 3, \dots$$



Note: R_i can only take values from the set $\{0, \frac{1}{m}, \frac{2}{m}, \dots, \frac{m-1}{m}\}$.

So technically it is **discrete**, not continuous. But if m is huge (like 2^{31}), the gaps are tiny and nobody notices.

Example 1: Let Us Generate Some Numbers!

Setup: $X_0 = 27$, $a = 17$, $c = 43$, $m = 100$.

Step 1: Start with seed $X_0 = 27$.

Step 2: Compute X_1 :

$$X_1 = (17 \times 27 + 43) \bmod 100$$

$$= 502 \bmod 100 = 2 \quad \Rightarrow \quad R_1 = 2/100 = 0.02$$

Step 3: Compute X_2 :

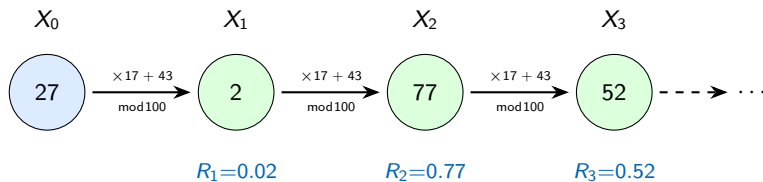
$$X_2 = (17 \times 2 + 43) \bmod 100 = 77 \bmod 100 = 77 \quad \Rightarrow \quad R_2 = 0.77$$

Step 4: Compute X_3 :

$$X_3 = (17 \times 77 + 43) \bmod 100 = 1352 \bmod 100 = 52 \quad \Rightarrow \quad R_3 = 0.52$$

And so on! Each number feeds into the next.

Visualizing the Process



Observations

- Each X_i **completely determines** X_{i+1} . No actual randomness!
- Change the **seed** X_0 and you get a different sequence.
- The quality depends entirely on the choices of a, c, m .

Since X_i can only take values in $\{0, 1, 2, \dots, m - 1\}$:

- There are at most m distinct values.
- Eventually, the sequence **must repeat**.
- Once it repeats a value, the **entire sequence repeats from that point**.

The Period P

The **period** is the length of the sequence before it starts repeating.

We want P to be as **large as possible**.

Ideally, $P = m$ (every possible integer appears exactly once before repeating).

A short period is a disaster. Imagine your simulation reuses the same “random” numbers over and over.

Example 2: Seeing the Period in Action

Setup

$$X_{i+1} = 13X_i \pmod{64}, \quad c = 0$$

This is a multiplicative linear congruential generator with different initial seeds.

Observed periods

Seed	Period
$X_0 = 4$	4
$X_0 = 2$	8
$X_0 = 1$	16
$X_0 = 3$	16

Lesson

Odd seeds produce the maximum period of 16, whereas even seeds produce shorter periods. Thus, the choice of seed significantly affects the generated sequence.

i	$X_0 = 1$	$X_0 = 2$	$X_0 = 3$	$X_0 = 4$
0	1	2	3	4
1	13	26	39	52
2	41	18	59	36
3	21	42	63	20
4	17	34	51	4
5	29	58	23	
6	57	50	43	
7	37	10	47	
8	33	2	35	
9	45		7	
10	9		27	
11	53		31	
12	49		19	
13	61		55	
14	25		11	
15	5		15	
16	1		3	

Empty entries indicate that the sequence has already returned to its initial seed.

Conditions for Maximum Period

The choice of a , c , m , and X_0 determines the period.

Case 1: $m = 2^b$, $c \neq 0$ (mixed congruential):

- Maximum period $P = m = 2^b$ when c and m are relatively prime, and $a = 1 + 4k$ for some integer k .

Case 2: $m = 2^b$, $c = 0$ (multiplicative):

- Maximum period is only $P = m/4 = 2^{b-2}$.
- Achieved when X_0 is **odd** and $a = 3 + 8k$ or $a = 5 + 8k$ for $k = 0, 1, 2, \dots$

Case 3: m is **prime**, $c = 0$:

- Maximum period $P = m - 1$.
- Requires: the smallest integer k such that $a^k - 1$ is divisible by m equals $k = m - 1$ (the smallest k is $m - 1$).

Why Period 16 in Example 2?

Let us verify: $a = 13$, $m = 2^6 = 64$, $c = 0$.

This is Case 2 ($m = 2^b$, $c = 0$). Maximum period $= m/4 = 64/4 = 16$. ✓

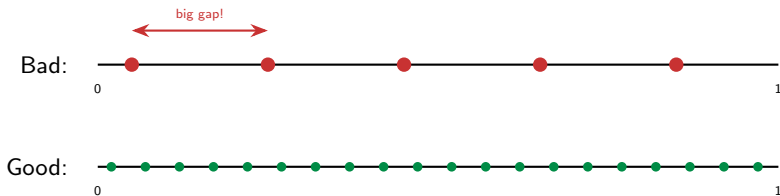
Check the multiplier: $a = 13 = 5 + 8(1)$, which is of the form $5 + 8k$. ✓

Check the seeds:

- $X_0 = 1$ (odd) $\Rightarrow$ period 16 ✓
- $X_0 = 3$ (odd) $\Rightarrow$ period 16 ✓
- $X_0 = 2$ (even) $\Rightarrow$ period 8 < 16
- $X_0 = 4$ (even) $\Rightarrow$ period 4 < 16

Maximum Density

Density = how closely the generated R_i values fill $[0, 1]$.



Key Insight

A large modulus m means more possible values for R_i , which means smaller gaps. Modern generators use $m = 2^{31} - 1 \approx 2.1 \times 10^9$. Gap size $\approx 4.7 \times 10^{-10}$. Practically continuous!

The Modulo Trick: Why Powers of 2?

Computers use **binary** (base 2). When $m = 2^b$, computing $X \bmod 2^b$ is trivially fast — just **keep the last b bits!**

Decimal analogy (Example 3): Let $m = 10^2 = 100$, $X_0 = 63$, $a = 19$, $c = 0$.

$$X_1 = 19 \times 63 \bmod 100 = 1197 \bmod 100 = \mathbf{97}$$

$$X_2 = 19 \times 97 \bmod 100 = 1843 \bmod 100 = \mathbf{43}$$

$$X_3 = 19 \times 43 \bmod 100 = 817 \bmod 100 = \mathbf{17}$$

Notice: $\bmod 100$ just means “keep the last 2 digits.”

Similarly, $\bmod 2^b$ means “keep the last b bits.”

Result: the modulo step costs essentially **nothing** on a binary computer.

A Real-World Generator (Example 4)

This generator has been **extensively tested** and is widely used:

$$a = 7^5 = 16,807, \quad c = 0, \quad m = 2^{31} - 1 = 2,147,483,647$$

Note: m is a **prime number**. Period: $P = m - 1 > 2$ billion!

Let us generate: Seed $X_0 = 123,457$.

$$X_1 = 16807 \times 123457 \bmod (2^{31} - 1) = \mathbf{2,074,941,799}$$

$$R_1 = X_1 / 2^{31} = \mathbf{0.9662} \text{ (dividing by } m + 1, \text{ for large } m, m \text{ or } m + 1 \text{ effect negligible)}$$

$$X_2 = \mathbf{559,872,160}, \quad R_2 = \mathbf{0.2607}$$

$$X_3 = \mathbf{1,645,535,613}, \quad R_3 = \mathbf{0.7662}$$

Testing Random Numbers

Trust, But Verify

We have a generator. It produces numbers.

They *look* random.

But are they?

Trust, But Verify

How do we check?

We need **statistical tests**.

(Because “they look fine to me” is not a valid statistical argument.)

Two Categories of Tests

We test the two properties we care about:

Uniformity Tests

Are the numbers evenly spread across $[0, 1]$?

- Kolmogorov-Smirnov test
- Chi-square test

Independence Tests

Are the numbers free of patterns?

- Autocorrelation test
- Runs test
- Serial test

Important Caveat

Passing all tests does **not guarantee** randomness — some pattern might go undetected. But **failing** a test is a definite red flag.

Setting Up Our Tests

For uniformity:

$H_0 : R_i \sim \text{Uniform}[0, 1]$ (numbers are uniform)

$H_1 : R_i \not\sim \text{Uniform}[0, 1]$ (numbers are NOT uniform)

For independence:

$H_0 : R_i$ are independent

$H_1 : R_i$ are NOT independent

The **significance level** α controls how strict we are:

$$\alpha = P(\text{reject } H_0 \mid H_0 \text{ is actually true})$$

Typical: $\alpha = 0.05$ means 5% chance of wrongly rejecting a good generator.

Multiple Testing

Example: Suppose we perform 20 independent hypothesis tests, each at significance level $\alpha = 0.05$.

Assume that **all 20 null hypotheses are true**. What is the probability of obtaining at least one statistically significant result purely by chance?

$$\begin{aligned}P(\text{at least one significant result by chance}) &= 1 - P(\text{no significant results}) \\&= 1 - (1 - 0.05)^{20} \\&\approx 0.64\end{aligned}$$

Interpretation

Even though all 20 null hypotheses are true, there is approximately a **64% chance** that at least one test will appear statistically significant purely by chance.

Multiple Testing: How Bad Does It Get?

For k independent tests at significance level α , assuming that all null hypotheses are true,

$$P(\text{at least one significant result by chance}) = 1 - (1 - \alpha)^k.$$

Number of tests k	$P(\text{at least one false positive})$
1	0.05
5	0.23
10	0.40
20	0.64
50	0.92
100	0.994

Key Message

The 5% significance level applies to each individual test. As the number of tests increases, the probability of obtaining at least one significant result purely by chance (incorrectly rejecting at least one true H_0) also increases.

What Does This Mean for Testing an RNG?

Situation 1: Many tests on one RNG output

An RNG is usually evaluated using several different tests.

A test is reported as **failed** when

$$p < \alpha,$$

meaning that its null hypothesis is rejected.

Suppose a correct RNG is examined using 10 independent tests, each with $\alpha = 0.05$. Then

$$P(\text{at least one test fails by chance}) = 1 - (0.95)^{10} \approx 0.40.$$

Interpretation

Even a correct RNG has about a 40% chance of failing at least one of the 10 tests purely by chance. Therefore, one isolated failure does not prove that the RNG is defective.

What Does This Mean for Testing an RNG?

Situation 2: Repeating a test on many fresh sequences

Suppose the same test is applied to 100 independent sequences generated by a correct RNG, using $\alpha = 0.05$.

For each sequence, there is a 5% chance of rejection even when the RNG is correct. Therefore, the expected number of chance rejections is

$$100 \times 0.05 = 5.$$

- Around 5 failures out of 100 may be normal.
- Many more than 5 failures may indicate a weakness.
- Repeated failure of the same test is more concerning than one isolated failure.

Practical message

One isolated failed test may be caused by chance. Consistent failures across fresh sequences, or failures in several different tests, provide stronger evidence that the RNG has a real problem.

Frequency Test 1: Kolmogorov-Smirnov (K-S)

Goal: Compare our generated numbers to the theoretical $U(0, 1)$ distribution.

Key concepts:

- The **theoretical CDF** of $U(0, 1)$: $F(x) = x$ for $0 \leq x \leq 1$.
- The **empirical CDF** from sample $R_1, R_2, \dots, R_N$:

$$S_N(x) = \frac{\text{number of } R_i\text{'s } \leq x}{N}$$

The Idea

If the numbers are truly uniform, $S_N(x)$ should be close to $F(x)$. The K-S test measures the **largest gap** between them:

$$D = \max_x |F(x) - S_N(x)|$$

If D is too large $\Rightarrow$ reject H_0 .

K-S Test: The Procedure

Given a sample $R_1, R_2, \dots, R_N$:

1. **Sort** the data: $R_{(1)} \leq R_{(2)} \leq \dots \leq R_{(N)}$.
2. Compute $D^+ = \max_{1 \leq i \leq N} \left(\frac{i}{N} - R_{(i)} \right)$
3. Compute $D^- = \max_{1 \leq i \leq N} \left(R_{(i)} - \frac{i-1}{N} \right)$
4. Test statistic: $D = \max(D^+, D^-)$.
5. Look up the **critical value** D_α from the K-S table (depends on N and α).
6. **Decision:**
 - If $D > D_\alpha$: **Reject** H_0 (evidence of non-uniformity!).
 - If $D \leq D_\alpha$: **Fail to reject** H_0 .

Example 6: K-S Test Worked Out

Data: 0.44, 0.81, 0.14, 0.05, 0.93. $N = 5, \alpha = 0.05$.

Step 1: Sort:

$$R_{(1)} = 0.05, \quad R_{(2)} = 0.14, \quad R_{(3)} = 0.44, \quad R_{(4)} = 0.81, \quad R_{(5)} = 0.93.$$

Step 2–3: Compute the candidate values for D^+ and D^- .

i	$R_{(i)}$	$\frac{i}{N}$	$D_i^+ = \frac{i}{N} - R_{(i)}$	$\frac{i-1}{N}$	$D_i^- = R_{(i)} - \frac{i-1}{N}$
1	0.05	0.20	0.15	0.00	0.05
2	0.14	0.40	0.26	0.20	-0.06
3	0.44	0.60	0.16	0.40	0.04
4	0.81	0.80	-0.01	0.60	0.21
5	0.93	1.00	0.07	0.80	0.13

$$D^- = \max_i \left(R_{(i)} - \frac{i-1}{N} \right) = \mathbf{0.21}.$$

Step 4: For $\alpha = 0.05$ and $N = 5$, $D_{0.05} = 0.565$.

Step 5: Since $0.26 < 0.565$, **do not reject H_0** . There is no significant evidence that the generated numbers are non-uniform.

Largest observed gaps

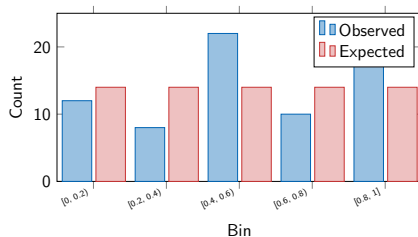
$$D^+ = 0.26, \quad D^- = 0.21.$$

Therefore,

$$D = \max(D^+, D^-) = \mathbf{0.26}.$$

Frequency Test 2: Chi-Square Test

Idea: Divide $[0, 1]$ into n equal bins. Count how many numbers fall in each bin. Compare to what we expect.



If the bars look similar $\Rightarrow$ probably uniform. If wildly different $\Rightarrow$ probably not uniform.

Chi-Square: The Formula

The test statistic is:

$$\chi_0^2 = \sum_{i=1}^n \frac{(O_i - E_i)^2}{E_i}$$

- O_i = **observed** count in bin i
- E_i = **expected** count = N/n (for uniform distribution)
- n = number of bins, N = total observations

Intuition: Each term $\frac{(O_i - E_i)^2}{E_i}$ measures how “surprised” we are by bin i .

- If $O_i \approx E_i$: small contribution. (Not surprised.)
- If O_i is far from E_i : large contribution. (Very surprised!)

Compare χ_0^2 to the critical value $\chi_{\alpha, n-1}^2$. If $\chi_0^2 > \chi_{\alpha, n-1}^2$: **reject** H_0 .

Example 7: Chi-Square Test

Data: 100 random numbers. $n = 10$ bins, $\alpha = 0.05$. Expected: $E_i = 100/10 = 10$.

Interval	O_i	E_i	$(O_i - E_i)^2$	$(O_i - E_i)^2 / E_i$
[0.0, 0.1)	8	10	4	0.4
[0.1, 0.2)	8	10	4	0.4
[0.2, 0.3)	10	10	0	0.0
[0.3, 0.4)	9	10	1	0.1
[0.4, 0.5)	12	10	4	0.4
[0.5, 0.6)	8	10	4	0.4
[0.6, 0.7)	10	10	0	0.0
[0.7, 0.8)	14	10	16	1.6
[0.8, 0.9)	10	10	0	0.0
[0.9, 1.0]	11	10	1	0.1
Total	100	100		$\chi_0^2 = \mathbf{3.4}$

Critical value: $\chi_{0.05,9}^2 = 16.9$. Since $3.4 \ll 16.9$: **Do not reject H_0** . Looks fine!

K-S vs. Chi-Square: Which to Use?

- Both the K-S and chi-square are acceptable for uniformity testing, provided that the sample size is large.
- K-S is generally more powerful, can be applied to small sample sizes.
- Chi-square is valid only for large samples, say $N \geq 50$.

But Uniformity Tests Are Not Enough!

A sequence like 0.01, 0.02, ..., 0.09, 0.11, 0.12, ... is perfectly uniform but **completely predictable**. We also need to test for **independence**!

The Autocorrelation Problem

Consider this sequence (read left to right):

0.12	0.01	0.23	0.28	0.89	0.31	0.64	0.28	0.83	0.93
0.99	0.15	0.33	0.35	0.91	0.41	0.60	0.27	0.75	0.88
0.68	0.49	0.05	0.43	0.95	0.58	0.19	0.36	0.69	0.87

Looks random if we only check the overall distribution.

But every 5th number starting from position 5 is large:

0.89, 0.93, 0.91, 0.88, 0.95, 0.87

Key idea

The sequence may still look uniform, but the **order** may not be random. Autocorrelation tests check whether numbers separated by a fixed lag are related.

Autocorrelation Test: The Setup

We choose:

i = starting position, ℓ = lag

Then we look at the subsequence

$$R_i, R_{i+\ell}, R_{i+2\ell}, \dots, R_{i+(M+1)\ell}.$$

Here, M is the largest integer such that

$$i + (M + 1)\ell \leq N.$$

Interpretation

We are not testing all numbers at once. We are testing whether each selected number is related to the next selected number:

$$(R_i, R_{i+\ell}), (R_{i+\ell}, R_{i+2\ell}), \dots$$

It Should be a Two-tailed Test

For a good random-number sequence, we want:

$$\rho_{i\ell} = 0$$

meaning no relationship between numbers separated by lag ℓ .

But dependence can happen in two directions:

Case	Meaning	Problem?
$\rho_{i\ell} > 0$	Average is higher than expected	yes
$\rho_{i\ell} < 0$	Average is lower than expected	yes
$\rho_{i\ell} = 0$	no clear relationship	desired

Why two-tailed?

Because both large positive and large negative autocorrelation indicate non-random dependence. So we reject if $\hat{\rho}_{i\ell}$ is too far from zero in either direction.

Average Product of Neighboring Pairs

For each neighboring lag-pair, multiply the two numbers: $R_{i+k\ell}R_{i+(k+1)\ell}$.

If two numbers are independent $U(0, 1)$ random numbers, then $E[R] = \frac{1}{2}$.

So for two independent numbers R and R' , $E[RR'] = E[R]E[R'] = \frac{1}{2} \cdot \frac{1}{2} = \mathbf{0.25}$.

Therefore, under independence, the average product should be close to 0.25.

$$\hat{\rho}_{i\ell} = \frac{1}{M+1} \sum_{k=0}^M R_{i+k\ell} R_{i+(k+1)\ell} - 0.25$$

Why Subtract 0.25?

The value 0.25 is the “normal” average product when the numbers are independent.

Example: if the sequence is truly random, we expect

$$\frac{1}{M+1} \sum_{k=0}^M R_{i+k\ell} R_{i+(k+1)\ell} \approx 0.25.$$

So

$$\hat{\rho}_{i\ell} = \text{average product} - 0.25 \approx 0.$$

Meaning of $\hat{\rho}_{i\ell}$

- $\hat{\rho}_{i\ell} \approx 0$: no evidence of dependence
- $\hat{\rho}_{i\ell} > 0$: products are larger than expected
- $\hat{\rho}_{i\ell} < 0$: products are smaller than expected

We Also Need $\sigma_{\hat{\rho}_{i\ell}}$

Even for a truly random sequence, $\hat{\rho}_{i\ell}$ will not be exactly zero.

So we ask:

Is the observed $\hat{\rho}_{i\ell}$ unusually far from 0?

To answer that, we need the standard deviation of $\hat{\rho}_{i\ell}$:

$$\sigma_{\hat{\rho}_{i\ell}} = \frac{\sqrt{13M+7}}{12(M+1)}.$$

Intuition

This measures how much random fluctuation we expect in $\hat{\rho}_{i\ell}$ when there is actually no autocorrelation.

Some Background

Let $R \sim U(0, 1)$.

This means that every value between 0 and 1 is equally likely, with probability density

$$f_R(r) = 1, \quad 0 \leq r \leq 1.$$

The expected value of R is

$$E[R] = \int_0^1 r f_R(r) dr = \int_0^1 r dr = \left[\frac{r^2}{2} \right]_0^1 = \frac{1}{2}.$$

We will also need the second moment:

$$E[R^2] = \int_0^1 r^2 f_R(r) dr = \int_0^1 r^2 dr = \left[\frac{r^3}{3} \right]_0^1 = \frac{1}{3}.$$

Two useful values

$$\boxed{E[R] = \frac{1}{2}}, \quad \boxed{E[R^2] = \frac{1}{3}}.$$

These values will be used to calculate the variance and covariance of the products.

Variance of One Product

Define one pair-product as

$$Y_k = R_{i+k\ell} R_{i+(k+1)\ell}.$$

To make the notation easier, write

$$Y_k = AB,$$

where A and B are independent $U(0, 1)$ random numbers.

Its expected value is

$$E[Y_k] = E[AB] = E[A]E[B] = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}.$$

Its variance is

$$\text{Var}(Y_k) = E[Y_k^2] - (E[Y_k])^2.$$

Since $Y_k^2 = A^2 B^2$ and A, B are independent,

$$E[Y_k^2] = E[A^2 B^2] = E[A^2]E[B^2] = \frac{1}{3} \cdot \frac{1}{3} = \frac{1}{9}.$$

Therefore,

$$\text{Var}(Y_k) = \frac{1}{9} - \left(\frac{1}{4}\right)^2 = \frac{1}{9} - \frac{1}{16} = \boxed{\frac{7}{144}}.$$

Why Does Covariance Appear?

The estimator uses several products:

$$Y_0, Y_1, \dots, Y_M.$$

Although the original random numbers are independent, adjacent products share one random number.

For example, let

$$Y_k = AB, \quad Y_{k+1} = BC,$$

where A, B, C are independent $U(0, 1)$ random numbers.

Both products contain B , so Y_k and Y_{k+1} are not independent. Their covariance is

$$\text{Cov}(Y_k, Y_{k+1}) = E[Y_k Y_{k+1}] - E[Y_k]E[Y_{k+1}].$$

$$\text{Now, } Y_k Y_{k+1} = (AB)(BC) = AB^2C.$$

$$\text{Using the independence of } A, B, \text{ and } C, E[AB^2C] = E[A]E[B^2]E[C] = \frac{1}{2} \cdot \frac{1}{3} \cdot \frac{1}{2} = \frac{1}{12}.$$

$$\text{Also, } E[Y_k]E[Y_{k+1}] = \frac{1}{4} \cdot \frac{1}{4} = \frac{1}{16}.$$

$$\text{Therefore, } \text{Cov}(Y_k, Y_{k+1}) = \frac{1}{12} - \frac{1}{16} = \boxed{\frac{1}{48}}.$$

Combining the Variance and Covariance

Recall that $\hat{\rho}_{i\ell} = \frac{1}{M+1} \sum_{k=0}^M Y_k - 0.25$.

The constant 0.25 does not affect variance, so

$$\text{Var}(\hat{\rho}_{i\ell}) = \text{Var}\left(\frac{1}{M+1} \sum_{k=0}^M Y_k\right).$$

For a sum of possibly dependent variables, $\text{Var}(\sum Y_k) = \sum \text{Var}(Y_k) + 2 \sum_{j < k} \text{Cov}(Y_j, Y_k)$.

Define $S = \sum_{k=0}^M Y_k$.

Then $\hat{\rho}_{i\ell} = \frac{1}{M+1} S - 0.25$.

The constant -0.25 does not affect variance: $\text{Var}(\hat{\rho}_{i\ell}) = \text{Var}\left(\frac{1}{M+1} S\right)$.

For any constant c , $\text{Var}(cS) = c^2 \text{Var}(S)$.

Here, $c = \frac{1}{M+1}$, so

$$\text{Var}(\hat{\rho}_{i\ell}) = \left(\frac{1}{M+1}\right)^2 \text{Var}(S).$$

How Many Variance Terms Are There?

Since, $\text{Var}(\sum Y_k) = \sum \text{Var}(Y_k) + 2 \sum_{j < k} \text{Cov}(Y_j, Y_k)$.

and every Y_k has $\text{Var}(Y_k) = \frac{7}{144}$, we have

$$\begin{aligned} \sum_{k=0}^M \text{Var}(Y_k) &= \underbrace{\frac{7}{144} + \frac{7}{144} + \cdots + \frac{7}{144}}_{M+1 \text{ terms}} \\ &= \boxed{(M+1) \frac{7}{144}}. \end{aligned}$$

How Many Covariance Terms Are There?

Only adjacent products have nonzero covariance:

$$(Y_0, Y_1), (Y_1, Y_2), \dots, (Y_{M-1}, Y_M).$$

The adjacent pairs are indexed by $k = 0, 1, \dots, M-1$, there are M adjacent pairs.

Each adjacent pair has $\text{Cov}(Y_k, Y_{k+1}) = \frac{1}{48}$.

Thus,

$$\sum_{k=0}^{M-1} \text{Cov}(Y_k, Y_{k+1}) = \underbrace{\frac{1}{48} + \frac{1}{48} + \dots + \frac{1}{48}}_{M \text{ terms}} = M \frac{1}{48}.$$

But the formula contains

$$2 \sum_{j < k} \text{Cov}(Y_j, Y_k),$$

so the covariance contribution is $\boxed{2M \frac{1}{48}}$.

Let's Put the Two Parts Together

We have found that

$$\text{Var} \left(\sum_{k=0}^M Y_k \right) = \underbrace{(M+1) \frac{7}{144}}_{M+1 \text{ variance terms}} + \underbrace{2M \frac{1}{48}}_{M \text{ adjacent covariance terms}} .$$

Including the constant factor $1/(M+1)^2$, we have

$$\text{Var}(\hat{\rho}_{i\ell}) = \frac{1}{(M+1)^2} \left[(M+1) \frac{7}{144} + 2M \frac{1}{48} \right] .$$

Simplifying,

$$\boxed{\text{Var}(\hat{\rho}_{i\ell}) = \frac{13M+7}{144(M+1)^2} .}$$

Obtaining the Standard-Deviation Formula

The standard deviation is the square root of the variance:

$$\sigma_{\hat{\rho}_{i\ell}} = \sqrt{\text{Var}(\hat{\rho}_{i\ell})}.$$

Using

$$\text{Var}(\hat{\rho}_{i\ell}) = \frac{13M + 7}{144(M + 1)^2},$$

We obtain,

$$\sigma_{\hat{\rho}_{i\ell}} = \frac{\sqrt{13M + 7}}{12(M + 1)}.$$

Now the Test Statistic Z_0

The null hypothesis says:

$$H_0 : \rho_{i\ell} = 0.$$

So the expected value of $\hat{\rho}_{i\ell}$ under H_0 is zero.

The test statistic is

$$Z_0 = \frac{\hat{\rho}_{i\ell} - 0}{\sigma_{\hat{\rho}_{i\ell}}} = \frac{\hat{\rho}_{i\ell}}{\sigma_{\hat{\rho}_{i\ell}}}.$$

Meaning of Z_0

Z_0 tells us how many standard deviations $\hat{\rho}_{i\ell}$ is away from zero.

For $\alpha = 0.05$, reject H_0 if

$$|Z_0| > 1.96.$$

Example 8: Autocorrelation Test

Test: 3rd, 8th, 13th, ... numbers. $i = 3$, $\ell = 5$, $N = 30$, $\alpha = 0.05$.

Find M : largest integer where $3 + (M + 1) \cdot 5 \leq 30$. So $M = 4$.

Subsequence: $R_3, R_8, R_{13}, R_{18}, R_{23}, R_{28} = 0.23, 0.28, 0.33, 0.27, 0.05, 0.36$

Compute $\hat{\rho}_{35}$:

$$\hat{\rho}_{35} = \frac{1}{5} [(0.23)(0.28) + (0.28)(0.33) + (0.33)(0.27) + (0.27)(0.05) + (0.05)(0.36)] - 0.25$$

$$= \frac{1}{5} [0.0644 + 0.0924 + 0.0891 + 0.0135 + 0.018] - 0.25 = 0.0555 - 0.25 = \mathbf{-0.1945}$$

Standard deviation: $\sigma = \frac{\sqrt{13(4)+7}}{12(5)} = \frac{\sqrt{59}}{60} = \mathbf{0.1280}$

Test statistic: $Z_0 = \frac{-0.1945}{0.1280} = \mathbf{-1.52}$

Since $|-1.52| = 1.52 < 1.96 = z_{0.025}$: **Do not reject H_0** . No evidence of autocorrelation.

Autocorrelation: A Warning About Fishing

In a dataset of $N = 30$ numbers, there are *many* possible subsequences to test.

If you test 10 subsequences with $\alpha = 0.05$:

$$P(\text{finding autocorrelation by chance}) = 1 - (0.95)^{10} \approx 0.40$$

The Fishing Problem

If you look hard enough for patterns, you may find them — even in truly random data.

(Finding shapes in clouds.)

Bottom line: A few rejections out of many tests are expected. Worry only if there are *significantly more* rejections than α would predict.

Monte Carlo Simulation

Now that we can **generate** and **test**
random numbers...

What can we actually **do** with them?

Monte Carlo Simulation

A scheme employing random numbers to solve stochastic *or* deterministic problems.

(Yes, you can use randomness to solve non-random problems.
That is the beautiful part.)

Why “Monte Carlo”?

The name comes from the **Monte Carlo Casino** in Monaco.

The method was developed during **World War II** by scientists working on the atomic bomb (Manhattan Project).

Stanislaw Ulam and **John von Neumann** invented this method.

The Integration Problem

Suppose we want to evaluate:

$$I = \int_a^b g(x) dx$$

where $g(x)$ is some function that we **cannot integrate analytically**.

Standard approaches: Simpson's rule, trapezoidal rule, etc. — work great in 1D. But in **high dimensions** (multiple integrals), these methods struggle badly.

The Key Insight: Integration as Expectation

Let $X \sim U(a, b)$ (cont. rand. variable distributed uniformly on $[a, b]$). Define $Y = (b - a) \cdot g(X)$.

The expected value of Y :

$$\begin{aligned} E(Y) &= (b - a) \cdot E[g(X)] \\ &= (b - a) \int_a^b g(x) \cdot \underbrace{f_X(x)}_{=1/(b-a)} dx \\ &= (b - a) \cdot \frac{1}{b - a} \int_a^b g(x) dx \\ &= \int_a^b g(x) dx = \boxed{I} \end{aligned}$$

The integral equals the expected value of a random variable!

We have converted integration into an estimation problem.

So How Do We Actually Estimate I ?

We showed that $I = E(Y)$. Great. But how do you **estimate** an expected value?

The same way you estimate anything in statistics:

Take many samples and average them.

Monte Carlo integration does the same thing.

Large numbers at work:

the average of many random samples converges to the true expected value.

The Sample Mean Estimator

Our estimator for $I = E(Y)$ is the **sample mean**:

$$\bar{Y}(n) = \frac{1}{n} \sum_{i=1}^n Y_i$$

where each $Y_i = (b - a) \cdot g(X_i)$ and $X_1, X_2, \dots, X_n$ are **independent** $U(a, b)$ random variables.

Substituting:

$$\bar{Y}(n) = (b - a) \cdot \frac{1}{n} \sum_{i=1}^n g(X_i)$$

In plain English:

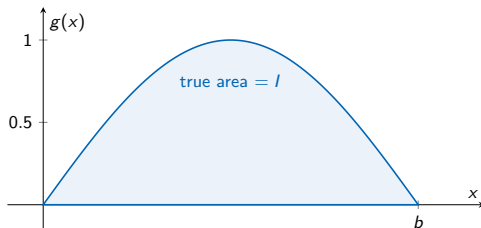
1. Pick n random points $X_1, \dots, X_n$ uniformly in $[a, b]$.
2. Evaluate g at each point: $g(X_1), g(X_2), \dots, g(X_n)$.
3. Average those values: $\frac{1}{n} \sum g(X_i)$.
4. Multiply by the interval length $(b - a)$.

The Geometric Picture

Think about what $\frac{1}{n} \sum g(X_i)$ actually computes: the **average height** of $g(x)$ at random points.

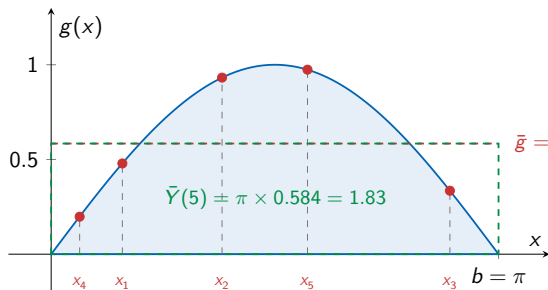
Now multiply by $(b - a)$:

$$\bar{Y}(n) = \underbrace{(b - a)}_{\text{base}} \times \underbrace{\frac{1}{n} \sum g(X_i)}_{\text{average height}} = \text{area of a rectangle}$$



The rectangle approximates the area under the curve. As we add more sample points, the average height converges to the true average of $g(x)$, and the rectangle area converges to I .

Visualizing the Random Sampling



Step 1: Pick random x -values in $[0, \pi]$. Evaluate $\sin(X_i)$ at each (red dots).

Step 2: Average the heights: $\bar{g} = (0.479 + 0.932 + 0.335 + 0.198 + 0.974)/5 = 0.584$.

Step 3: Multiply by $(b - a) = \pi$: estimate = 1.83. True answer = 2. Not bad for 5 points!

Let Us Try It: $\int_0^\pi \sin x \, dx$ Step by Step

We know the exact answer: $\int_0^\pi \sin x \, dx = [-\cos x]_0^\pi = -(-1) - (-1) = 2$.

With $n = 10$: Generate 10 random values $X_i \sim U(0, \pi)$:

$$\begin{array}{lllll} X_1 = 0.52 & X_2 = 1.19 & X_3 = 2.83 & X_4 = 0.21 & X_5 = 1.77 \\ X_6 = 2.50 & X_7 = 0.89 & X_8 = 1.53 & X_9 = 2.97 & X_{10} = 2.11 \end{array}$$

Evaluate $\sin(X_i)$ at each:

$$\begin{array}{lllll} 0.497 & 0.929 & 0.301 & 0.208 & 0.980 \\ 0.599 & 0.777 & 0.999 & 0.172 & 0.870 \end{array}$$

Average: $\frac{1}{10}(0.497 + 0.929 + \cdots + 0.870) = 0.633$

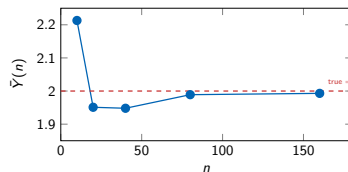
Multiply by $(b - a) = \pi$: $\bar{Y}(10) = \pi \times 0.633 = \mathbf{1.989}$

True value = 2. Error $\approx 0.5\%$. With just 10 random points!

Does More Sampling Help?

Let us increase n and watch the estimate improve:

n	$\bar{Y}(n)$	Error
10	2.213	0.213 (10.7%)
20	1.951	0.049 (2.5%)
40	1.948	0.052 (2.6%)
80	1.989	0.011 (0.6%)
160	1.993	0.007 (0.4%)



The estimate “orbits” around the true value, getting closer and tighter. This is the law of large numbers in action.

Why Is the Error Not Monotonically Decreasing?

Notice something odd: $\bar{Y}(40) = 1.948$ is *worse* than $\bar{Y}(20) = 1.951$.

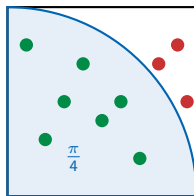
This is **not** a bug. It is the nature of randomness.

Each time we run the experiment, the random X_i 's are different. So $\bar{Y}(n)$ is itself a **random variable**.

Monte Carlo: Estimating π (A Classic)

Problem: Estimate π using only random numbers.

Idea: A quarter circle of radius 1 fits inside a unit square.



Unit square

Method: Generate random points (x, y) in the unit square. Count how many fall inside the quarter circle ($x^2 + y^2 \leq 1$).

$$\hat{\pi} = 4 \times \frac{\text{points inside circle}}{\text{total points}}$$

Estimating π : Step by Step

Step 1: Generate n pairs (X_i, Y_i) where $X_i, Y_i \sim U(0, 1)$.

Step 2: For each pair, check if $X_i^2 + Y_i^2 \leq 1$.

Step 3: Count the “hits” (inside the circle). Call this h .

Step 4: Estimate $\pi \approx 4h/n$.

Why does this work?

$$P(\text{point inside quarter circle}) = \frac{\text{area of quarter circle}}{\text{area of square}} = \frac{\pi/4}{1} = \frac{\pi}{4}$$

So $h/n \approx \pi/4$, hence $\pi \approx 4h/n$.

n (points)	$\hat{\pi}$
100	3.24
1,000	3.16
10,000	3.1436
100,000	3.1412
1,000,000	3.14178

True: $\pi = 3.14159 \dots$

